

第三讲实验指南

把每一个输出追溯到执行步骤

这次实验的目标不是多打印几个结果，而是获得一种检查程序的方法：先预测某段代码应该产生什么，再运行、对照；若不一致，回到最早出现分歧的一行，查清是数据、类型、顺序、索引还是表达式改变了问题。最终你应独立写出三候选的训练比较，而不必背诵所有 Python 语法。

建议分两轮。第一轮按 Notebook 的 13 个代码单元顺序运行，每格先在纸上预写关键输出。第二轮重启内核，修改指定的一处代码，验证自己能解释结果如何变化。数据仍为第一讲合成房屋，所有测试标签公开，仅用来核对计算与流程。

一 环境与文件

需要 Python 3.10 或更新版本。experiment.py 仅使用标准库中的 math，没有第三方依赖、网络下载或 GPU 要求。experiment.ipynb 需要 Jupyter，已有 Anaconda 环境通常可以提供。实际制作时所测版本及执行方式在 verification.json 中记录；这不代表已在所有操作系统或所有 Anaconda 版本实装测试。

打开终端，切换到本讲目录，运行：

```
python --version
python experiment.py
```

终端应逐段显示表达式、变量状态、训练表和最后指标，末尾出现 PASS: 15 explicit checks。这里使用 print 输出，保证终端脚本和交互环境都能看到相同的关键信息。仅看到 PASS 仍不足以完成学习验收，你还要能够解释至少一个检查为什么写在那里。

Notebook 路线是打开 experiment.ipynb，从第一个代码单元向下执行。不要把终端命令 python experiment.py 当作 Python 表达式写进普通代码格。第一次运行前选择 Python 3 内核，最终验收用重启内核后全部运行。若执行次序混乱，先重启而不是继续猜当前变量是什么。

本讲 CSV 放在 data/hand_calculation.csv，供核对纸笔数据。为了不提前引入文件读写，脚本和 Notebook 把这些同样的数值直接写成列表。修改 CSV 不会改变当前程序输出；若你改了 CSV 却期待输出变化，正好说明需要先检查程序实际读取了什么。

二 建一张预测输出记录表

在纸上画四列：单元编号、运行前预测、真实输出、解释差异。每格选一个关键输出，不必把屏幕所有内容抄下来。例如第 2 格记录“只改 area 后 prediction 是否更新”，第 7 格记录“错误累加器最后得到多少”。先写预测，再点执行，才能区分自己理解了过程还是只是在事后接受答案。

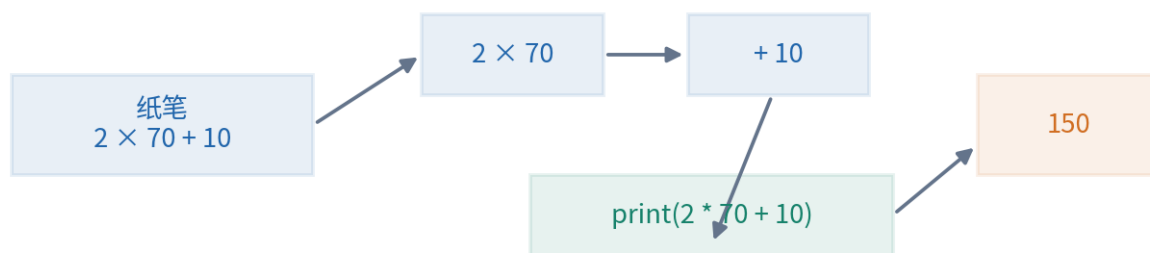


图 1 用同一个表达式核对纸笔与程序。第一轮只改变括号，其他数字保持不变，观察的是计算结构而不是模型优劣。

如果预测错了，保留错的答案并补一句解释，不要直接擦掉。这个记录能帮助你发现稳定的误解：是否总把赋值当公式、是否把最后一个索引当长度、是否忘记循环内会覆盖变量。学习日志的价值在于暴露错误机制，而不只是留一页看起来全对的结果。

三 表达式和赋值的两次对照

第 1 格输出 150 与 160。第一条先算 $2 * 70$ ，再加 10；第二条先算括号内 $70 + 10$ ，再乘 2。解释器没有偏向哪一种，它只是执行你写下的运算结构。选择哪种才符合面积模型，要回到任务定义。

第 2 格先将 area 设为 70，得到 prediction 为 150；之后只把 area 改成 80，打印旧 prediction 仍为 150。重新执行 `prediction = weight * area + bias` 后，才变成 170。请手画三行状态表：每行记录 area、weight、bias、prediction 在执行后的值。

做一个改变：把 `area = 80` 移到第一次计算 prediction 之前，先预测第一次打印会是多少，再运行。应为 170。程序并没有“记住你本来想用 70”，它只使用执行到那一行时名字所指向的值。

另一个改变是在重启后的新内核中新建一个临时代码格，只输入 `print(prediction)`，不要先运行任何定义 prediction 的代码。应得到名称尚未定义的错误。这个错误说明依赖没有建立，不应通过随意执行某个旧单元碰运气来隐藏它。将所有依赖按顺序写在文档中，才是可重跑方案。

四 类型和括号检查

第 3 格显示 int、float、str，分别对应整数、浮点数、字符串；随后显示 75 与 705。后者是字符串连接结果，在 `print` 时引号通常不显示，所以光看屏幕数字外观无法判断类型。若不确定，要同时查看 type，而不是相信数字长得像什么。

自己添加 `print(float("70.5") + 5)`，应得到 75.5。再在一个临时格中试 `print("70.5" + 5)`，会因为文本与整数不能按这种方式相加而报 `TypeError`。保留一份错误截图或抄下错误类型即可；不要把故意失败的格子留在最终“全部运行应成功”的验收 Notebook 里。

本格正确平均值是 7.5，缺少括号的结果是 18.75。检查步骤是先把代码中的操作顺序改写成中文，再与 MAE 定义比较。不要因为结果也是正数、也有小数点，就认为它一定是某种合法误差。

乘方例子输出 4 与 -4。以后写平方误差时，把整个 `prediction - label` 放入括号再平方。这条习惯能避免一种代码可运行、却完全改变损失的错误。数学符号抄到程序里时，不能省掉语言要求的结构。

五 用索引走过四条记录

第 4 格应显示长度 4、第一条 50 与 110、最后一条 90 与 190，中间切片为 [60,80]。现在用纸笔列出合法非负索引 0、1、2、3；最后索引为 3，长度为 4。让这两个概念并排出现，比只背“Python 从零开始”更有效。

把 `prices` 改为 [190,170,130,110]，暂时不要重跑整份正式实验。先判断 B 的四个预测仍是多少，新的绝对误差又是多少。预测仍为 [110,130,170,190]，误差变为 [80,40,40,80]，MAE 为 60。这个数字变化来自配对被破坏，而不是 B 的参数或公式变了。

恢复原列表，再进入后面步骤。建议复制一个临时格做破坏实验，完成后删除该格或保留在文字说明中，最终从头运行的主流程继续使用原始合成数据。否则你可能一边读着原数据说明，一边实际上在运行修改后的标签。



图 2 同索引必须指向同样本。面积和值的个数相同并不足以保证配对正确，单独修改一列的顺序会改变实际任务。

六 区分共享修改与独立实验副本

第 5 格把 `other_name` 关联到 `original`，再通过 `other_name` 修改首个元素，所以 `original` 也变成 [55,60,80,90]。随后 `copy` 创建另一个列表，修改 `independent` 的首元素为 999，`original` 仍保持 [55,60,80,90]。

注意训练用的 `areas` 是另一次列表创建，值仍为 [50,60,80,90]。名字 `original` 听上去很像“永不改变的原始数据”，但名称本身不会阻止修改。保护数据需要实际操作和检查，不能依靠给变量取一个令人安心的名字。

做一次验证：在一个临时格打印 `areas`，确认它保持原样；再画出 `original`、`other_name`、`independent` 各自指向哪个列表。图中两支箭头指向同一个对象，正是共享修改的原因。不要把变量当成各自独立、自动复制内容的盒子。

七 跟踪 C 的四轮状态

第 6 格输出如下轨迹。索引 0 时预测 115、误差 5、累计 5；索引 1 时 130、0、5；索引 2 时 160、10、15；索引 3 时 175、15、30。最后 MAE 为 $30 / 4 = 7.5$ 。

先只阅读循环体，找出“当前值”与“累计值”。`prediction` 和 `error` 每轮覆盖，`total_error` 保留前面总和后再加当前误差，`candidate_predictions` 和 `absolute_errors` 则每轮把一个新值附加到列表末尾。不同存储方式对应不同目的，不能互相替代。

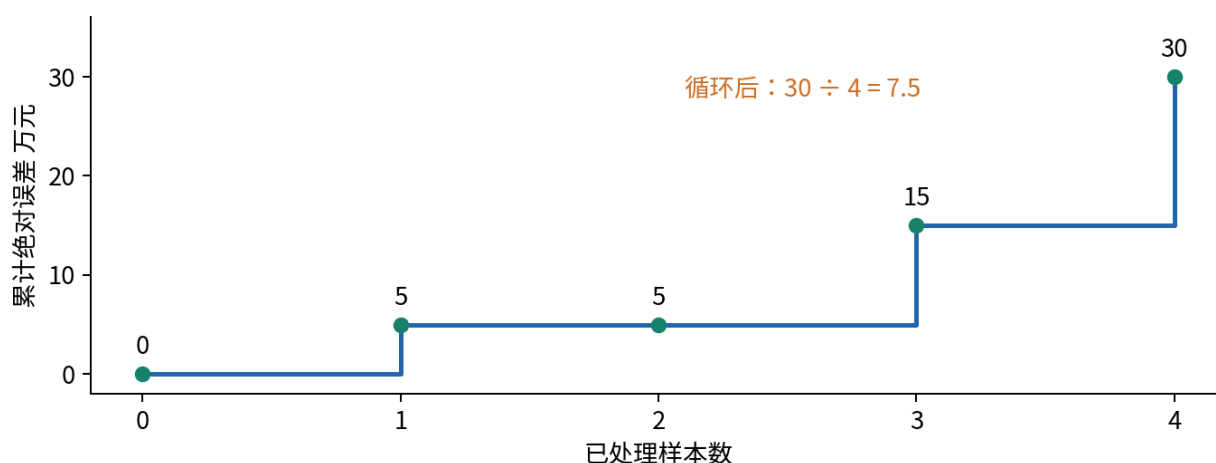


图 3 从当前误差到累计误差。最后除以 4 的前提，是累计中确实包含了四个样本，而不是最后一条。

第 7 格故意把 `wrong_total = 0` 放在循环内部。先预测它得到 3.75，再运行确认。它不报错，是因为每条操作在语言上都合法；它错在每轮丢掉了历史总和。使用“循环结束后总和应等于已处理误差之和”这个性质，可以在第二轮就发现问题，不必等最后分数很奇怪。

八 三候选比较的检查点

第 8 格应打印三个候选的预测列表与 MAE。A 为 [100,120,160,180] 与 10；B 为 [110,130,170,190] 与 0；C 为 [115,130,160,175] 与 7.5。对照纸笔表时先检查预测，再检查绝对误差，最后看平均值。

如果 A 正确、B 和 C 偏大，优先检查 `total_error` 是否在每个候选开始时清零。如果每个候选只留下一个预测，检查 `predictions = []` 是否错误地放入了样本循环。如果三个候选使用同一斜率，检查 `weights` 的索引是否用了 `candidate`，而不是 `index` 或固定位置。

这些“优先检查”不是保证错误只会在那里，而是基于症状缩小范围的方法。最小输入最容易查：一个候选、一个样本时应该做一次乘加、一次绝对值；扩展到两个样本，再到四个样本；最后才扩展到三个候选。一次加太多变化，会让错误来源互相遮挡。

第 9 格选择 B、2、10。对一个临时分数列表 [2,2,5]，手动分别用 `<` 和 `<=` 重写选择循环，核对前者保留第一项，后者选中第二项。每种行为都来自明确规则，不是随机发生。

九 冻结后只做预测与评价

第 10 格将选中的参数固定，按两个测试面积得到 [140,180]。第 11 格才出现标签 [144,174]，算出模型 MAE 5 和固定训练均值基线 MAE 15。代码位置表达信息顺序，但公开教学文件不提供真实保密保证。

做一个思考，不需要改正式程序：如果测试标签换成 [1000,1000]，但训练数据和候选不变，`best_index` 应改变吗？不应改变。测试结果会变差，但训练选择应该仍为 B。若你的选择随这种修改改变，说明程序把最终评价信息引回了开发步骤。

另一个思考是把测试面积换成 [300,400]。计算公式仍能给出 610、810，但它们已经远离训练面积范围。程序会算不等于预测可信。范围外输入恰恰说明编程正确和模型适用性需要分开评价。

十 浮点实验做多精确才够

第 12 格打印 0.30000000000000004、False 和一个约为 $5.55e-17$ 的差值，指定容差比较则为 True。记住 $1e-12$ 是十进制科学计数法写法，不是一个神秘指令。它表示允许的差异尺度，仍需依据任务决定。

把输入改成 $0.5 + 0.25$ ，与 0.75 比较，应得到精确相等。这说明并非凡是小数都不能精确表示。再试 `sum([0.1] * 10)` 与 1.0，观察累积运算是否逐位相等。不要用一次例子推出“浮点永远错误”或“只要保留两位就安全”。

如果某项测量本身只精确到一万元，数值计算误差在 10^{-12} 万元附近通常不是首要问题；如果某个算法会放大很小的差异，就需要另外分析稳定性。容差应与数值来源和后续计算联系起来，而不是固定套一个足够小的数。

十一 运行检查与边界输入

最后一格包含 15 条 `assert`。这里 `assert` 的意思是要求后面的条件成立；若不成立，程序停止并提示 `AssertionError`。这里按普通 python 命令运行，不使用会去掉 `assert` 的 `-O` 优化选项；`assert` 也不能代替生产环境的输入安全检查。完整测试组织在下一讲学习，但你已经能读懂 `scores == [10.0,0.0,7.5]` 这样的要求。参见 [Python 官方 assert 说明](#)。

其中一条检查 `areas` 仍为原始列表，另一条检查错误累加器确实得到 3.75。这两条看起来不像“最终性能指标”，却能帮助暴露共享修改和误解程序的风险。测试既可以要求正确实现给出已知答案，也可以要求反例确实展示所说的错误。

单条样本应该如何计算？若 `areas` 为 `[70]`，`prices` 为 `[150]`，`B` 的 MAE 为 0。将标签改成 `[155]`，它的 MAE 应为 5。空数据应该如何处理？平均值分母为零，因此任务未定义；第 4 讲会让函数主动拒绝它，而不是把“没观察到错误”写成零误差。

长度不等的两列表也不能继续按原逻辑相信结果。如果面积比标签多，访问末尾时可能报错；如果面积比标签少，按面积长度循环可能直接忽略多出的标签而不报错。因此“有没有错误信息”不是充分标准，应明确检查输入长度是否匹配。

十二 提交一份短实验记录

记录 Python 版本、运行方式、是否重启后完整运行；列出三候选预测与 MAE；保留一个赋值状态表和一个累计状态表；描述两个故障，其中至少一个是没有报错的逻辑错误；最后写清这仍然是合成小数据上的计算核验。

可以用一句话概括所学：“我可以通过执行顺序、配对与累计状态，把最终分数追溯到每一行输入和每一步计算。”如果只能背出 `B` 的分数是 0，却不能解释 `total_error` 放在哪里，就还需要重做第七节。

十三 加练与详解

问题一：先执行 `x = 4`，`y = x`，再执行 `x = 9`，`y` 是否变成 9？解答：不会，整数名字重新关联不改变 `y` 指向的旧整数 4。把它与共享列表的修改比较，区分重新赋值一个名字和原地修改一个可变对象。

问题二：列表 `[3,4,5]` 的索引 -1、0、2 分别是什么？解答：5、3、5。-1 与 2 在这个长度为 3 的列表中指向同一个末尾位置，并不意味着它们在所有长度列表中都等价。

问题三：在 `C` 的计算中，若只遍历 `range(3)`，得到误差和 15。如果仍除以 4，结果为 3.75；若除以 3，结果为 5。哪一个为四行训练数据的 MAE？解答：都不是。第一个分母与实际处理数不一致，第二个只是在前三行上的平均。先检查评价对象，再讨论公式。

问题四：某人把训练 MAE `[10,0,7.5]` 误写成 `[0.10,0,0.075]`，候选仍选 `B`，能否说程序已正确？解答：不能。排序碰巧没变，数值单位或缩放已经改变。独立核对中间值能发现“最终选择正确”掩盖的问题。

问题五：想让程序输出两位小数，是否应该在每次累计后 `round(total,2)`？解答：通常不应把显示需求混进计算。先按所需精度完成计算，再在最终展示阶段格式化；若任务确实要求每步离散化或货币分单位计算，应明确写入算法定义并分析影响。

问题六：为什么同一个 Notebook 只点击 Run All 有时还不够？解答：旧内核可能保留文件里已经删掉、但曾经执行过的变量或导入。先重启清空状态，再全部运行，才能更可靠地检查文档是否自包含。它也仍不等同于跨平台所有环境都验证过。

本指南的工具语义依据与核验日期见正文参考资料及 `source-checks.json`。